Unidad 4: Del problema al primer modelo de Machine Learning

4.1. Planteando el problema: ¿existe una relación en los datos?

4.1.1. La primera pregunta: ¿podemos encontrar una relación entre los datos?

Ya construimos una base conceptual que ahora nos permitirá dar un paso importante. Ya sabemos qué significa que una máquina aprenda a partir de datos, conocemos la diferencia entre clasificación y regresión y comprendemos que un modelo necesita variables que describan adecuadamente aquello que queremos estudiar.

Hasta ahora, sin embargo, hablamos principalmente desde una perspectiva conceptual. Sabemos **qué** queremos hacer.

Ahora necesitamos comenzar a comprender **cómo** puede hacerlo matemáticamente un algoritmo. Para introducir esta idea vamos a utilizar un problema deliberadamente sencillo.

Supongamos que una empresa inmobiliaria posee información histórica sobre viviendas que fueron vendidas durante los últimos años. Para cada propiedad conocemos su superficie y el precio final de venta.

La empresa quiere responder una pregunta aparentemente simple:

> ***¿Cuánto podría valer una vivienda de 120 metros cuadrados?***

- Tenemos entonces información histórica.
- Tenemos una variable que describe cada vivienda.
- Tenemos una respuesta conocida para los ejemplos anteriores.

Y queremos utilizar esos ejemplos para realizar una predicción sobre una vivienda que todavía no conocemos. Estamos nuevamente frente a un problema de **aprendizaje supervisado**. Pero existe una diferencia importante respecto de muchos ejemplos que analizamos anteriormente.

Aquí la respuesta que queremos obtener es un número.No queremos determinar si una vivienda pertenece a una categoría, queremos estimar su precio. Por lo tanto, estamos frente a un problema de **regresión**.

Ahora aparece la pregunta central de esta clase:

> ***¿Cómo podemos encontrar matemáticamente una relación entre la superficie de una vivienda y su precio?***

4.1.2. Mirar los datos antes de construir el modelo

Antes de utilizar cualquier algoritmo podemos comenzar observando los datos.

Supongamos que disponemos de las siguientes propiedades:

Superficie	Precio
50 m^2	US$65.000
60 m^2	US$72.000
75 m^2	US$86.000
90 m^2	US$102.000
110 m^2	US$123.000
130 m^2	US$142.000
150 m^2	US$160.000

Si representamos estos valores gráficamente, probablemente observemos algo interesante. A medida que aumenta la superficie también aumenta el precio.

Los puntos no se encuentran exactamente sobre una misma línea, porque el precio de una propiedad depende de muchas otras variables: ubicación, estado de conservación, antigüedad, cantidad de habitaciones, cochera, calidad de construcción, etc.

Sin embargo, aparece una tendencia, y precisamente esa tendencia es la que nos interesa capturar. Podríamos intentar dibujar una línea que atraviese aproximadamente el centro de los puntos.

La pregunta es:

¿Podemos utilizar esa línea para realizar predicciones?

La respuesta es sí.

Y esa es la idea fundamental detrás de la **Regresión Lineal**.

4.2. La recta como primer modelo

4.2.1. La recta como modelo

Una recta puede expresarse matemáticamente mediante una ecuación muy conocida:

$$y=mx+b$$

Probablemente muchos estudiantes hayan visto esta expresión durante sus estudios de matemáticas.

En Machine Learning vamos a darle una interpretación diferente. Ya no será simplemente una ecuación que debemos resolver. Será un **modelo**.

Podemos pensar que:

- x representa una característica de entrada.
- y representa el valor que queremos predecir.
- m representa cuánto cambia y cuando cambia x.
- b representa el punto donde la recta intercepta el eje vertical.

En el ejemplo inmobiliario:

x=superficie e y=precio

Entonces la recta representa una aproximación matemática de la relación existente entre ambas variables.

4.2.2. ¿Qué significa "aprender" una recta?

Aquí aparece uno de los conceptos más importantes.

Cuando decimos que un modelo de Machine Learning **aprende**, no significa que la computadora memorice los ejemplos. Significa que encuentra los parámetros de una función que permiten representar adecuadamente la relación existente en los datos.

En una regresión lineal simple tenemos dos parámetros principales:

m y **b**

El algoritmo necesita encontrar cuáles son los valores de m y b que producen la recta que mejor representa los datos disponibles.

Supongamos que encontramos:

m=1.000 y **b=15.000**

Nuestro modelo sería:

$$\hat{y}=1000x+15000$$

Entonces, para una vivienda de 100 m^2:

$$\hat{y}=1000(100)+15000$$

El modelo produciría una predicción de:

US$115.000.

Observemos algo fundamental. El algoritmo no encontró mágicamente el precio de esa vivienda. Construyó una función a partir de ejemplos anteriores y utilizó esa función para realizar una

estimación. Esto es exactamente lo que significa **generalizar**. El modelo aprendió una relación a partir de datos conocidos y posteriormente la aplicó sobre un caso que no había visto.

4.2.3. Predicción no significa certeza

Aquí debemos detenernos porque aparece una diferencia fundamental entre una predicción de Machine Learning y una verdad matemática.

Si calculamos:

2+2 obtenemos exactamente 4, no estamos realizando una predicción, estamos resolviendo una operación matemática. En cambio, si nuestro modelo estima que una vivienda vale US$115.000, ese valor no representa necesariamente el precio real de venta.

- Podría venderse por US$110.000.
- Podría venderse por US$120.000.

Incluso podría venderse por US$95.000 debido a alguna característica que nuestro modelo no contempla. Por eso hablamos de **predicción** o **estimación**, el modelo no conoce el futuro, utiliza información histórica para producir una aproximación. Esta distinción es fundamental cuando trabajamos con Inteligencia Artificial. Un modelo puede ser extremadamente bueno y aun así equivocarse.

La pregunta profesional no es:

> *"¿El modelo se equivoca?"*

Todos los modelos reales cometen errores.

La pregunta correcta es:

> ***"¿Qué tan grandes son sus errores y resultan aceptables para el problema que queremos resolver?"***

Esta pregunta nos llevará directamente al próximo concepto fundamental.

4.2.4. Parámetros

Los parámetros son valores internos que el modelo aprende durante el proceso de entrenamiento.

Volvamos a la ecuación:

y=wx+by=wx+b

Los valores ww y bb son parámetros del modelo.

No somos nosotros quienes decidimos manualmente cuál debe ser su valor final. El proceso de entrenamiento los ajusta utilizando los datos.

Esta característica diferencia a los parámetros de los hiperparámetros.

Es importante que el estudiante pueda responder una pregunta aparentemente sencilla:

¿Quién determina el valor de un parámetro?

La respuesta general es: el proceso de entrenamiento.

Cuando entrenamos una regresión lineal, por ejemplo, el algoritmo determina los coeficientes que mejor se ajustan a los datos según el criterio utilizado.

Cuando entrenamos una red neuronal, los pesos y bias se modifican durante el entrenamiento.

Cuando entrenamos otros modelos, los parámetros pueden adoptar formas diferentes.

Por eso no debemos pensar que todos los modelos tienen los mismos parámetros. Cada familia de modelos posee su propia estructura matemática.

4.2.5 Hiperparámetros

Los hiperparámetros, en cambio, son configuraciones que establecemos antes o durante el proceso de entrenamiento y que controlan cómo se construye o aprende el modelo.

Por ejemplo, en KNN debemos decidir el valor de KK, que determina cuántos vecinos se utilizan para realizar una predicción.

En un árbol de decisión podemos establecer una profundidad máxima.

En Random Forest podemos definir cuántos árboles utilizar.

En una red neuronal podemos decidir la tasa de aprendizaje, la cantidad de épocas o el tamaño del batch.

Estos valores no son aprendidos de la misma manera que los parámetros internos.

Podemos resumir la diferencia:

Concepto	¿Quién lo determina?	Ejemplo
Parámetro	El entrenamiento	pesos de una red, coeficientes
Hiperparámetro	Configuración del entrenamiento	K, profundidad, learning rate

Esta diferencia será especialmente importante cuando lleguemos a la experimentación. En Machine Learning no basta con entrenar un modelo una sola vez. Muchas veces debemos probar distintas configuraciones para determinar cuál funciona mejor.

4.3. Midiendo el error del modelo

4.3.1. El error: medir qué tan mal predice nuestro modelo

Supongamos que conocemos el precio real de una vivienda ,el propietario la vendió finalmente por:

US$120.000

Nuestro modelo había predicho:

US$115.000

Tenemos entonces una diferencia de:

US$5.000

Ese valor representa el error de nuestra predicción y podemos expresarlo conceptualmente como:

$$\text{error} = \text{valor real} - \text{valor predicho}$$

En este caso:

$$\text{error} = 120000 - 115000$$

El resultado es:

$$5000$$

Nuestro modelo se quedó corto en US$5.000.

Ahora imaginemos otra propiedad cuyo precio real fue US$100.000 y nuestro modelo predijo US$105.000.

El error sería:

$100000 - 105000 = -5000$

Aparece un detalle interesante, un error puede ser positivo o negativo. El signo indica la dirección del error.

Un error positivo significa que el modelo predijo por debajo del valor real, y un error negativo significa que predijo por encima. Pero si queremos medir simplemente **qué tan grande fue el error**, probablemente no nos interese demasiado el signo.

Esto nos lleva a la necesidad de definir una función que permita medir sistemáticamente el desempeño del modelo. Y aquí comienza una de las ideas matemáticas centrales del Machine Learning.

4.3.2 La función de pérdida

Supongamos que tenemos diez viviendas y calculamos el error cometido por nuestro modelo en cada una.

Podríamos obtener algo como:

+3.000, -5.000, +2.000, -1.500, +4.500, -3.200, +800, -2.100, +1.700, -4.000

Si simplemente sumamos todos los errores, algunos positivos y negativos podrían cancelarse. El resultado podría ser cercano a cero aunque el modelo estuviera cometiendo errores importantes.

Por eso necesitamos otra estrategia. Una de las más utilizadas consiste en elevar los errores al cuadrado. Así obtenemos siempre valores positivos:

error²

Esta idea conduce al **error cuadrático medio**, conocido como **MSE — Mean Squared Error**.

Conceptualmente podemos expresarlo como:

$$MSE=1/n \cdot \sum (yi - y\hat{}i)^2$$

En esta etapa es mucho más importante comprender qué significa, tenemos muchos ejemplos.

1. Para cada ejemplo calculamos la diferencia entre el valor real y el valor predicho.
2. Elevamos esa diferencia al cuadrado.
3. Finalmente calculamos el promedio.
4. El resultado nos proporciona una medida global del error cometido por el modelo.

Cuanto menor sea el MSE, mejor estará ajustando el modelo los datos de entrenamiento.

4.3.3. ¿Qué está buscando realmente el algoritmo?

Ahora podemos comenzar a responder una pregunta que al principio parecía abstracta.

¿Qué significa entrenar una regresión lineal?

Significa buscar los valores de los parámetros de la recta que minimicen el error definido por nuestra función de pérdida.

Dicho de una manera mucho más intuitiva:

- Tenemos muchos puntos.
- Probamos una recta.

- Medimos qué tan lejos quedan los puntos de esa recta.
- Calculamos el error.
- Movemos la recta.
- Volvemos a calcular el error.
- Continuamos hasta encontrar una configuración que produzca un error suficientemente pequeño según el criterio utilizado.

El algoritmo está realizando, en esencia, un problema de **optimización**.

Está buscando la mejor solución posible dentro del espacio de soluciones permitido por el modelo.

Este ciclo constituye uno de los grandes patrones conceptuales del Machine Learning.

4.3.4. Una pregunta antes de continuar

Supongamos lo siguiente:

> *Tenemos dos modelos para estimar el precio de una vivienda.*
>
> *El modelo A tiene un error promedio de US$8.000.*
> *El modelo B tiene un error promedio de US$15.000.*
>
> *¿Podemos afirmar automáticamente que el modelo A es mejor?*

La respuesta correcta es: **No necesariamente.**

- ¿Por qué?

Porque todavía necesitamos conocer cómo fueron obtenidos esos errores.

- ¿Se calcularon sobre los mismos datos?
- ¿Sobre datos de entrenamiento o datos nuevos?
- ¿Utilizamos la misma métrica?
- ¿Ambos modelos fueron evaluados bajo las mismas condiciones?

Esta pregunta nos permite recordar un concepto que comenzamos a estudiar con anterioridad:

Un modelo no debe evaluarse únicamente sobre los datos que utilizó para aprender.

Y aquí aparece nuevamente la importancia de separar los datos para entrenamiento y evaluación.

4.3.5. El problema de memorizar los datos

Imaginemos que entregamos a un estudiante las preguntas exactas que aparecerán en un examen.

El estudiante puede memorizar las respuestas, cuando llegue el examen, probablemente obtendrá una calificación excelente. Pero ¿significa eso que realmente aprendió?

- No necesariamente.
- Podría simplemente haber memorizado las respuestas.

En Machine Learning ocurre exactamente el mismo problema.

Un modelo puede conseguir un error extremadamente bajo sobre los datos utilizados durante el entrenamiento y, sin embargo, producir predicciones muy malas sobre ejemplos nuevos.

Esto es lo que conocemos como **overfitting**.

La verdadera capacidad de un modelo no consiste en reproducir perfectamente los ejemplos que ya conoce.

Consiste en generalizar correctamente frente a situaciones nuevas.

Por eso, cuando construimos una regresión lineal, no debemos preguntarnos únicamente:

> *"¿Qué tan bien ajusta los datos que le entregamos?"*

También debemos preguntar:

> ***"¿Qué tan bien funciona cuando recibe datos que nunca vio?"***

Esta diferencia será fundamental cuando pasemos de la teoría a Python.

4.3.6. Data leakage

Supongamos que queremos predecir si un cliente comprará un producto.

Entre las variables disponibles aparece:

fecha_compra

Si utilizamos esa variable para predecir una compra que todavía no ocurrió, estamos introduciendo información que en realidad no estaría disponible en el momento de la predicción.

El modelo podría aprender una relación extraordinariamente buena porque estamos entregándole información del futuro.

Otro ejemplo sería utilizar una variable denominada:

monto_compra

para predecir si el cliente realizará esa misma compra.

El modelo tendría prácticamente la respuesta delante suyo.

El resultado podría parecer excelente durante la evaluación, pero sería completamente engañoso.

Esto es data leakage: información que no debería estar disponible para el modelo durante la predicción termina formando parte de las variables utilizadas para entrenarlo o evaluarlo.

La importancia de este concepto crecerá cuando dividamos los datos y construyamos nuestro flujo completo de entrenamiento.

4.4. De la teoría a un ejemplo concreto

4.4.1. Del modelo matemático al código

Hasta este momento hemos trabajado sin utilizar una biblioteca y esto es intencional.

Quiero que comprenda primero qué está haciendo el algoritmo antes de escribir:

```
model.fit(X, y)
```

Cuando posteriormente utilicemos Scikit-Learn, esa línea parecerá sencilla, pero detrás de ella existirán todos los conceptos que acabamos de estudiar.

- Tenemos:

X

que contiene nuestras variables de entrada.

- Tenemos:

y

que contiene el valor que queremos predecir.

- Tenemos un modelo.
- El modelo busca sus parámetros.
- Calcula predicciones.
- Compara esas predicciones con los valores reales.
- Calcula un error.

Y mediante un procedimiento matemático obtiene los parámetros que mejor representan los datos según el criterio elegido.

La biblioteca nos permite realizar todo este proceso de forma extremadamente sencilla. Pero la facilidad de uso de la herramienta no debe hacernos olvidar la matemática que existe detrás.

Ese será uno de los principios de esta asignatura.

> ***No queremos aprender únicamente a utilizar las librerías. Queremos entender qué están haciendo las librerías por nosotros.***

4.4.2. Un ejemplo real para cerrar esta primera parte

Volvamos al ejemplo inmobiliario.

Tenemos cientos de propiedades históricas.

Para cada una conocemos:

- superficie;
- cantidad de habitaciones;
- ubicación;
- antigüedad;
- cochera;
- precio final de venta.

Podríamos comenzar con un modelo extremadamente sencillo utilizando solamente la superficie.

El modelo aprende una relación entre:

superficie → precio

Y genera una recta, después podríamos incorporar nuevas variables. Entonces el modelo ya no trabajaría únicamente con una dimensión.

Comenzaría a utilizar:

superficie + habitaciones + antigüedad + ubicación + cochera → precio

La idea fundamental seguiría siendo la misma, lo que cambia es la cantidad de información disponible para realizar la predicción. Este ejemplo también nos permite observar una evolución natural.

Dividimos los datos en conjuntos destinados al entrenamiento, validación y prueba. A continuación seleccionamos un algoritmo de clasificación y realizamos el entrenamiento. El modelo aprende sus parámetros a partir del conjunto de entrenamiento. Luego realizamos predicciones sobre datos que no utilizamos directamente para ajustar esos parámetros. Evaluamos el desempeño.

Si los resultados no son satisfactorios, no necesariamente significa que "el algoritmo sea malo". Podemos revisar los datos, las variables, los hiperparámetros, la representación del problema o

incluso la definición del target. Si el modelo funciona bien, todavía debemos preguntarnos si generaliza adecuadamente y si la métrica utilizada realmente representa lo que necesita la empresa.

Finalmente, después de realizar los experimentos y seleccionar una configuración adecuada, podemos construir nuestro modelo final.

Más adelante estudiaremos otros algoritmos y finalmente llegaremos a modelos mucho más complejos, pero el principio fundamental permanecerá.

Utilizamos ejemplos conocidos para aprender una función que nos permita estimar resultados desconocidos.

4.4.3. Encontrar la recta que mejor representa los datos

Ya sabemos que una regresión lineal intenta encontrar una relación entre una o más variables de entrada y una variable numérica que queremos predecir. También vimos que, en el caso más sencillo, podemos representar esa relación mediante una recta de la forma:

$$\hat{y} = mx + b$$

La cuestión que todavía no resolvimos es probablemente la más importante: **¿cómo encuentra el algoritmo los valores de m y b?**

Esta pregunta nos lleva directamente al corazón matemático de la regresión lineal.

Imaginemos nuevamente el conjunto de viviendas que utilizamos anteriormente. Tenemos una serie de puntos distribuidos sobre un gráfico: en el eje horizontal colocamos la superficie y en el eje vertical el precio. Algunos puntos estarán relativamente cerca de una determinada tendencia y otros estarán un poco más alejados. Nuestro objetivo consiste en encontrar una recta que represente de la mejor manera posible esa distribución.

Podríamos dibujar una recta cualquiera. Por ejemplo, podríamos elegir una que pase por el centro aproximado de los datos. Pero inmediatamente aparecería una pregunta: **¿cómo sabemos que esa recta es mejor que otra?**

La respuesta consiste en medir el error.

Para cada vivienda podemos comparar el precio real con el precio que nuestra recta predice. Si la vivienda realmente se vendió por US$120.000 y nuestro modelo estima US$115.000, existe una diferencia de US$5.000. Si otra vivienda se vendió por US$90.000 y nuestro modelo estima US$96.000, tenemos un error de -US$6.000.

Si hacemos esto para todas las viviendas obtenemos una colección de errores. La recta que buscamos será aquella que, según el criterio matemático que hayamos definido, produzca el menor error global.

Aquí aparece una idea que acompañará prácticamente toda la asignatura: **un algoritmo de Machine Learning necesita alguna manera de determinar si está haciendo un buen trabajo o un mal trabajo**.

En regresión lineal, una de las formas clásicas de hacerlo consiste en utilizar el error cuadrático medio.

4.14. El error como distancia entre realidad y predicción

Podemos pensar visualmente el error de una predicción como la distancia vertical entre un punto real y la recta del modelo.

Supongamos que una determinada vivienda tiene una superficie de 100 m^2 y su precio real es US$120.000. Nuestra recta, para esos mismos 100 m^2, produce una predicción de US$112.000.

El punto real se encuentra por encima de la recta.

La distancia vertical entre ambos representa el error de esa predicción.

Si, en cambio, el precio real fuera US$105.000 y nuestra recta predijera US$112.000, el punto estaría por debajo de la recta.

El error tendría signo contrario.

Esta representación gráfica es extremadamente útil porque permite entender intuitivamente qué está intentando hacer el algoritmo. No está tratando de pasar exactamente por todos los puntos —algo que, en general, sería imposible o produciría un modelo poco útil—, sino de encontrar una posición de la recta que permita representar razonablemente el comportamiento general de los datos.

Podemos imaginar entonces que la recta se encuentra inicialmente en una posición cualquiera y que el algoritmo necesita desplazarla y modificar su inclinación hasta encontrar una configuración que reduzca el error total.

La pregunta siguiente es inevitable: **¿qué significa exactamente "reducir el error total"?**

4.15. El error cuadrático medio

Para responder esta pregunta necesitamos introducir formalmente una de las métricas más importantes de la regresión.

El **Mean Squared Error**, o **MSE**, calcula el promedio de los errores elevados al cuadrado.

Su expresión matemática es:

$$MSE=1/n \cdot \sum (yi - y\hat{}i)^2$$

No es necesario que en este momento el estudiante memorice la fórmula. Lo verdaderamente importante es comprender qué está haciendo.

Para cada observación tomamos el valor real yi, calculamos la diferencia con respecto al valor predicho y^i, elevamos esa diferencia al cuadrado y finalmente promediamos todos los resultados.

¿Por qué elevamos al cuadrado?

Hay dos razones importantes. La primera es evitar que los errores positivos y negativos se cancelen. La segunda es que los errores grandes reciben un peso considerablemente mayor.

Por ejemplo, supongamos dos errores:

$$2$$

y

$$10$$

Si simplemente los observamos, el segundo es cinco veces mayor que el primero. Pero cuando los elevamos al cuadrado obtenemos:

$$22=4$$

y

$$102=100$$

El segundo error pasa a pesar 25 veces más, esto significa que el MSE penaliza fuertemente las predicciones que se alejan mucho del valor real. Esta característica puede resultar útil cuando los errores grandes son especialmente problemáticos para nuestro problema.

4.16. Un ejemplo calculado paso a paso

Supongamos que nuestro modelo realiza predicciones sobre cuatro viviendas.

Vivienda	Precio real	Predicción
A	100.000	95.000
B	120.000	125.000
C	150.000	145.000
D	180.000	170.000

Calculamos primero los errores:

- $100000-95000=5000$
- $120000-125000=-5000$
- $150000-145000=5000$
- $180000-170000=10000$

Tenemos entonces:

$5000,-5000,5000,10000$

Si simplemente sumáramos estos valores, obtendríamos:

5000

Pero ese resultado no describe correctamente el comportamiento del modelo porque los errores positivos y negativos se compensaron parcialmente.

Por eso elevamos cada uno al cuadrado:

$$5000^2 \ (-5000)^2 \ 5000^2 \ 10000^2$$

y posteriormente calculamos el promedio. El resultado será el MSE correspondiente a esas cuatro predicciones.

Lo interesante de este procedimiento no es únicamente obtener un número. Ese número permite **comparar modelos**. Si construimos una segunda recta y obtenemos un MSE menor sobre el mismo conjunto de datos, podemos decir que esa segunda recta ajusta mejor esos datos según esta métrica.

4.17. Menor error no siempre significa mejor modelo

Acá conviene hacer una pausa porque aparece una trampa conceptual muy importante.

Supongamos que construimos dos modelos.

- El primero obtiene un MSE de 80.
- El segundo obtiene un MSE de 30.

A primera vista parece evidente que debemos elegir el segundo.

Pero todavía no sabemos **sobre qué datos** fueron calculados esos valores.

Si el modelo 1 obtuvo su resultado sobre datos que nunca había visto y el modelo 2 obtuvo el suyo sobre los mismos datos utilizados para entrenarlo, la comparación deja de ser válida.

Esto nos devuelve a una idea que ya apareció anteriormente: **la evaluación de un modelo debe realizarse utilizando información que permita medir su capacidad de generalización**.

Un modelo que simplemente memoriza los datos de entrenamiento puede presentar un error extremadamente bajo y aun así ser incapaz de realizar buenas predicciones sobre nuevos casos. Por eso, durante las próximas clases vamos a trabajar mucho con la separación entre entrenamiento y evaluación.

4.18. ¿Cómo se mueve la recta?

Ahora podemos volver a nuestra pregunta original. Tenemos una cantidad enorme de rectas posibles. Cada recta está determinada por dos parámetros:

m

y

b

El parámetro m determina la pendiente.

- Si m es positivo, la recta aumenta.
- Si m es negativo, disminuye.
- Si m es cercano a cero, la recta se vuelve prácticamente horizontal.

El parámetro b, por otro lado, determina el punto donde la recta intercepta el eje vertical.

- Modificar m cambia la inclinación.
- Modificar b desplaza la recta hacia arriba o hacia abajo.

Por lo tanto, podemos imaginar el proceso de entrenamiento como una búsqueda entre distintas combinaciones posibles de estos dos parámetros.

- Una combinación produce una determinada recta.
- Esa recta produce determinadas predicciones.
- Las predicciones generan determinados errores.
- Los errores producen un determinado valor de MSE.
- El algoritmo intenta encontrar los valores de m y b que minimizan ese error.

Esta es una manera extremadamente importante de comprender qué significa "entrenar".

Entrenar no es simplemente ejecutar una función. Entrenar significa encontrar parámetros que hagan que el modelo se comporte adecuadamente según una función de pérdida.

4.19. La pendiente tiene un significado real

Volvamos por un momento al ejemplo inmobiliario. Supongamos que después del entrenamiento obtenemos:

$$\hat{y}=1200x+10000$$

¿Qué significa el número 1200?

Representa la pendiente de la recta. En este contexto, podemos interpretarlo aproximadamente como que, según el modelo, cada metro cuadrado adicional se asocia con un incremento de US$1.200 en el precio estimado.

Esto es muy interesante porque la regresión lineal no solamente permite realizar predicciones, también puede ayudarnos a **interpretar relaciones entre variables**.

Supongamos que nuestro modelo estima:

$$\hat{y}=1200x+10000$$

Para una vivienda de 50 m^2:

$$1200(50)+10000=70000$$

Para una vivienda de 100 m^2:

$$1200(100)+10000=130000$$

Para una vivienda de 150 m^2:

$$1200(150)+10000=190000$$

Según este modelo, cada incremento de 50 m^2 genera aproximadamente US$60.000 adicionales en la predicción. Naturalmente, estamos simplificando muchísimo el problema. En el mundo real, el precio depende de muchas otras variables. Pero justamente por eso este ejemplo resulta tan útil para comenzar. Nos permite ver cómo una relación matemática puede transformarse en una herramienta de predicción.

4.5. Interpretando los coeficientes del modelo

4.5.1. ¿Y qué representa el término independiente?

El segundo parámetro de nuestra ecuación es:

b

Supongamos nuevamente:

$$\hat{y}=1200x+10000$$

El valor 10.000 corresponde a b. Matemáticamente representa el valor que tendría y cuando x=0.

Si reemplazamos:

x=0

obtenemos:

$$\hat{y}=10000$$

En determinados problemas esta interpretación puede tener sentido físico. En otros, puede ser simplemente un parámetro matemático necesario para posicionar correctamente la recta.

Esto también es importante para los estudiantes: **no todos los parámetros de un modelo necesariamente poseen una interpretación práctica directa**.Algunos representan relaciones muy intuitivas. Otros cumplen principalmente una función matemática dentro del modelo.

4.5.2. Regresión lineal con una variable y con muchas variables

Hasta ahora utilizamos un único predictor:

$$X = \text{superficie}$$

Por eso hablamos de una regresión lineal simple. Pero imaginemos ahora que queremos mejorar nuestro modelo inmobiliario.

Sabemos que el precio no depende únicamente de la superficie. También puede depender de la cantidad de habitaciones, la antigüedad, la ubicación y otros factores.

Entonces podemos incorporar múltiples variables.

La ecuación adopta una forma como:

$$\hat{y}=b+w_1x_1+w_2x_2+w_3x_3+\cdots+w_nx_n$$

Ahora tenemos varias características.

Por ejemplo:

x1 = superficie x2 = habitaciones x3 = antiguedad x4 = distancia al centro

Cada variable posee un peso diferente:

w1,w2,w3,w4

El modelo aprende esos pesos a partir de los datos. La idea esencial, sin embargo, sigue siendo exactamente la misma.

- Tenemos entradas.
- Tenemos una salida.
- Tenemos parámetros.
- Tenemos predicciones.
- Tenemos errores.

Y buscamos parámetros que minimicen esos errores. Esto nos prepara para una idea fundamental: **la regresión lineal puede extenderse de una recta en un plano a un hiperplano en espacios de mayor dimensión**.

No necesitamos todavía entrar en la geometría matemática de espacios multidimensionales. Lo importante es que el estudiante comprenda que la idea no cambia cuando aumentamos la cantidad de variables.

4.5.3. Una conexión importante

En las primeras clases hablamos de *features*, ahora podemos observar exactamente para qué sirven.

Las *features* son las variables x1,x2,…,xn que alimentan el modelo.

También hablamos de una variable objetivo.

- Ahora sabemos que esa variable se representa mediante y.

Hablamos de aprendizaje supervisado.

- Ahora podemos ver que el modelo recibe ejemplos donde conocemos tanto X como y.

Hablamos de entrenamiento.

- Ahora entendemos que entrenar significa encontrar los parámetros que permiten producir buenas predicciones.

Hablamos de generalización.

- Ahora sabemos que el verdadero objetivo no consiste en reproducir los datos de entrenamiento, sino en utilizar lo aprendido para predecir nuevos ejemplos.

De esta manera, la Regresión Lineal no aparece como un tema aislado. Es la **primera aplicación concreta de todos los conceptos que venimos construyendo desde el comienzo**.

4.5.4. La primera gran idea matemática de Machine Learning

Hay una idea que quiero que quede instalada porque volverá a aparecer una y otra vez:

Machine Learning puede verse como un problema de optimización.

- Tenemos un modelo con determinados parámetros.
- Esos parámetros producen predicciones.
- Las predicciones generan errores.
- Los errores se resumen mediante una función de pérdida.
- Y el algoritmo intenta encontrar los parámetros que minimicen esa pérdida.

En el caso de la regresión lineal, el modelo es relativamente sencillo y podemos comprender bastante bien qué está ocurriendo. Más adelante, cuando trabajemos con modelos mucho más complejos, las ecuaciones serán considerablemente más grandes y los espacios de búsqueda serán mucho más difíciles de visualizar, pero la lógica fundamental continuará siendo la misma, por eso vale la pena detenernos ahora y comprenderla profundamente.

4.5.5. Pregunta para pensar

Supongamos que tenemos dos rectas para predecir el precio de viviendas. La primera pasa cerca de la mayoría de los puntos, pero se aleja bastante de tres viviendas. La segunda se encuentra razonablemente cerca de casi todos los puntos, aunque no pasa exactamente por ninguno.

¿Cuál preferirían?

La respuesta que buscamos no debería ser simplemente "la segunda".

La discusión debería llevarnos a comprender que **un buen modelo no necesariamente pasa por todos los ejemplos**.

Lo que buscamos es capturar el patrón general.

Una recta que intenta adaptarse excesivamente a cada observación puede terminar perdiendo capacidad de generalización. Por eso el objetivo del aprendizaje no es conseguir una representación perfecta de los datos históricos. Es encontrar una representación suficientemente buena para producir predicciones útiles sobre datos nuevos.

4.6. Implementación en Python con Scikit-Learn

4.6.1. El momento de pasar a Python

Ahora sí estamos preparados para escribir nuestro primer modelo.

Hasta este momento hemos construido todo el razonamiento sin depender de ninguna biblioteca. Esto es deliberado. Cuando en la próxima sección aparezca Scikit-Learn y escribamos apenas unas pocas líneas de Python, el estudiante podrá comprender qué está sucediendo detrás de ellas.

Vamos a comenzar con un conjunto de datos pequeño, porque en esta primera experiencia no nos interesa esconder la matemática detrás de una enorme base de datos.

Podemos imaginar algo tan sencillo como:+

superficie = [50, 60, 75, 90, 110, 130, 150]

precio = [65000, 72000, 86000, 102000, 123000, 142000, 160000]

Tenemos nuestras características:

X

y nuestro objetivo:

y

Posteriormente utilizaremos:

```
from sklearn.linear_model import LinearRegression
```

y construiremos nuestro modelo.

Pero esta vez quiero que el estudiante sea capaz de mirar la instrucción:

```
modelo.fit(X, y)
```

y no verla como una fórmula mágica.

Debe poder decir:

> *"Estoy entregándole ejemplos conocidos al algoritmo para que encuentre los parámetros de una función lineal que minimicen el error."*

Ahí empieza realmente el Machine Learning aplicado.

4.6.2. De la teoría al código: construyendo nuestro primer modelo de regresión

Hasta este momento hemos estudiado la regresión lineal desde una perspectiva conceptual y matemática. Sabemos que nuestro objetivo consiste en encontrar una relación entre determinadas variables de entrada y un valor numérico que queremos predecir. También comprendimos que el modelo intenta encontrar los parámetros de una función que permita reducir el error de sus predicciones.

Ahora vamos a realizar el recorrido completo utilizando Python.

La intención de este ejemplo no es simplemente aprender a utilizar una biblioteca. De hecho, si solamente quisiéramos construir una regresión lineal, podríamos hacerlo en unas pocas líneas. Lo verdaderamente importante es que podamos mirar cada una de esas líneas y comprender **qué representa dentro del proceso de Machine Learning**.

Vamos a utilizar nuevamente el problema de las viviendas. Supongamos que una inmobiliaria posee información histórica de algunas propiedades y conoce tanto la superficie de cada vivienda como el precio al que finalmente fueron vendidas.

Podemos representar una pequeña muestra de esos datos de la siguiente manera:

superficie = [50, 60, 75, 90, 110, 130, 150]

precio = [65000, 72000, 86000, 102000, 123000, 142000, 160000]

Tenemos dos listas:

- La primera contiene nuestra variable de entrada: la superficie de la vivienda.
- La segunda contiene aquello que queremos predecir: el precio.

Es importante detenernos acá porque aparece nuevamente una distinción fundamental.

- La superficie constituye nuestra **feature**.
- El precio constituye nuestro **target**.

Estamos diciendo, en términos conceptuales:

> *"Utilizando la superficie de una vivienda, quiero aprender a estimar su precio."*

Todavía no hemos entrenado absolutamente nada, simplemente tenemos los datos.

4.6.3. ¿Por qué Scikit-Learn espera una estructura particular?

Para trabajar con Machine Learning utilizaremos **Scikit-Learn**, una de las bibliotecas fundamentales del ecosistema Python para aprendizaje automático. Una particularidad importante es que Scikit-Learn representa las variables de entrada mediante una estructura bidimensional.

Aunque nosotros pensemos que tenemos simplemente una lista de superficies:

[50, 60, 75, 90, 110, 130, 150]

para Scikit-Learn necesitamos representar esos datos como una matriz donde cada fila corresponde a una observación y cada columna corresponde a una característica.

Podemos hacerlo utilizando NumPy:

```
import numpy as np
X = np.array(superficie).reshape(-1, 1)
y = np.array(precio)
```

Ahora podemos imaginar X de esta manera:

[[50],

[60],

[75],

[90],

[110],

[130],

[150]]

Cada fila representa una vivienda, tenemos siete viviendas y una característica, por eso X tiene una dimensión de:

7 × 1

En cambio y contiene los siete precios. Esta diferencia entre X e y es extremadamente importante y conviene explicarla detenidamente a los estudiantes. Cuando trabajemos posteriormente con cinco, diez o cien características, la lógica será exactamente la misma. Las filas seguirán representando ejemplos y las columnas representarán variables.

Podemos incluso visualizar conceptualmente nuestra matriz:

Vivienda	Superficie
1	50
2	60
3	75
4	90
5	110
6	130

Vivienda	Superficie
7	150

Y tenemos separado el objetivo:

Vivienda	Precio
1	65.000
2	72.000
3	86.000
4	102.000
5	123.000
6	142.000
7	160.000

Esto nos permite introducir una regla que vamos a utilizar durante toda la asignatura:

> ***X representa la información que utilizamos para realizar la predicción; y representa aquello que queremos aprender a predecir.***

4.6.4. Creamos el modelo

Ahora podemos importar el algoritmo de regresión lineal:

```
from sklearn.linear_model import LinearRegression
```

Y posteriormente crear nuestro modelo:

```
modelo = LinearRegression()
```

Hasta acá todavía no aprendió absolutamente nada.

Esto es importante, cuando ejecutamos:

```
modelo = LinearRegression()
```

simplemente estamos creando un objeto que representa un modelo de regresión lineal.

Todavía

- No conoce nuestras viviendas.
- No conoce sus superficies.
- No conoce sus precios.
- No conoce la pendiente.

- No conoce el intercepto.

Es simplemente un modelo preparado para ser entrenado. Esto nos permite establecer una diferencia conceptual muy importante:

crear un modelo no es entrenarlo.

Crear el modelo significa seleccionar la estructura matemática que vamos a utilizar. Entrenarlo significa encontrar los parámetros adecuados utilizando los datos.

4.6.5. La línea que entrena el modelo

Ahora llegamos a una de las líneas más conocidas de Scikit-Learn:

```
modelo.fit(X, y)
```

Esta línea parece extremadamente sencilla, pero detrás de ella está ocurriendo todo lo que estudiamos durante las últimas páginas. Estamos entregando al algoritmo ejemplos conocidos.

Para cada vivienda conoce:

superficie → precio

El algoritmo analiza esos ejemplos y encuentra los parámetros que permiten construir la recta que mejor representa los datos según el criterio utilizado. En otras palabras, fit() realiza el proceso de entrenamiento.

Podemos pensar conceptualmente:

X + y
↓
ENTRENAMIENTO
↓
parámetros aprendidos
↓
modelo

Esta es una de las primeras líneas de código que los estudiantes deberían aprender a interpretar conceptualmente.

No quiero que memoricen:

> *"fit sirve para entrenar."*

Quiero que comprendan:

> ***"fit(X, y) recibe ejemplos conocidos y utiliza esos datos para aprender los parámetros del modelo."***

Esa diferencia parece pequeña, pero cambia completamente la forma de entender Machine Learning.

4.6.6. ¿Qué aprendió el modelo?

Después de ejecutar:

```
modelo.fit(X, y)
```

podemos preguntarle al modelo cuáles fueron los parámetros aprendidos.

Primero podemos consultar:

modelo.coef_

Este valor corresponde al coeficiente asociado a nuestra variable de entrada. En una regresión lineal simple corresponde a la pendiente m.

También podemos consultar:

modelo.intercept_

que corresponde al término independiente b.

Entonces podemos reconstruir conceptualmente la ecuación que aprendió el modelo:

$$\hat{y} = mx + b$$

Supongamos, simplemente a modo ilustrativo, que el modelo hubiera obtenido aproximadamente:

$$m = 1.000$$

y

$$b = 15.000$$

Entonces podríamos representar nuestro modelo como:

$$\hat{y} = 1000x + 15000$$

Esto significa que el modelo encontró una relación aproximada entre la superficie y el precio.

Y ahora aparece una cuestión fundamental. **La ecuación no fue escrita por nosotros.**

El algoritmo encontró esos parámetros a partir de los datos. Eso es precisamente lo que significa aprender.

4.31. Hagamos una predicción

Ahora llega el momento interesante, supongamos que aparece una vivienda nueva de:

120 m^2

La propiedad no estaba dentro de los datos utilizados durante el ejemplo. Queremos que nuestro modelo estime su precio. Podemos hacerlo de la siguiente manera:

```
nueva_vivienda = np.array([[120]])
prediccion = modelo.predict(nueva_vivienda)
print(prediccion)
```

El modelo devuelve un valor. Por ejemplo, podría producir algo cercano a:

[132000]

El número exacto dependerá de los datos utilizados y de los parámetros aprendidos. Pero conceptualmente acaba de ocurrir algo muy importante.

El modelo recibió:

120 m^2

y produjo:

precio estimado, no le dijimos cuál debía ser la respuesta, la obtuvo utilizando la relación aprendida durante el entrenamiento. Este es el momento en que podemos decir que el modelo está siendo utilizado para **generalizar**. Aprendió a partir de ejemplos anteriores y ahora intenta resolver un caso nuevo.

4.6.7. fit() y predict(): dos operaciones que debemos diferenciar

Tenemos:

```
modelo.fit(X, y)
```

y tenemos:

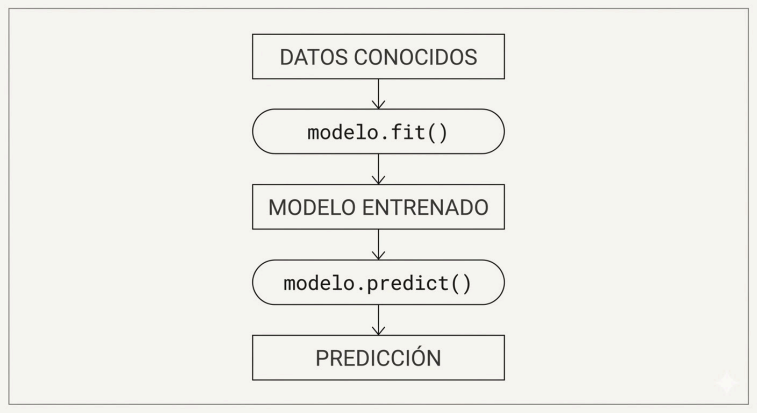
```
modelo.predict(X_nuevo)
```

No hacen lo mismo.

fit() corresponde al aprendizaje.

predict() corresponde a utilizar aquello que fue aprendido.

Podemos representarlo:



Esta separación conceptual será fundamental cuando trabajemos con clasificación, clustering y posteriormente con sistemas de recomendación.

4.6.8. ¿Cómo sabemos si nuestra predicción es buena?

Supongamos ahora que la vivienda de 120 m^2 finalmente fue vendida por:

US$135.000

Nuestro modelo había predicho:

US$132.000

Tenemos una diferencia de:

$135000 - 132000 = 3000$

El modelo se equivocó por aproximadamente US$3.000, pero ahora podemos hacernos una pregunta más interesante.

¿US$3.000 es mucho o poco?

La respuesta depende del contexto.

Para una vivienda cuyo valor ronda los US$130.000, un error de US$3.000 puede resultar relativamente pequeño. Para una aplicación financiera donde necesitamos una precisión de unos pocos cientos de dólares, podría ser demasiado grande.

Esto nos lleva nuevamente a una idea fundamental:

> ***No existe un modelo "bueno" en términos absolutos. Un modelo es bueno o malo en función del problema que intenta resolver y del error que resulta aceptable en ese contexto.***

No alcanza con preguntar:

> *"¿Qué accuracy tiene mi modelo?"*

Primero debemos saber **qué problema estamos resolviendo**.

En regresión, además, accuracy no es normalmente la métrica principal. Trabajaremos con métricas de error adecuadas al tipo de problema.

4.6.9. Visualizar lo que aprendió el modelo

Ahora tenemos una oportunidad excelente para conectar matemática, programación y visualización. Podemos representar las viviendas históricas como puntos, en el eje horizontal colocamos:

Superficie

y en el eje vertical:

Precio

Después dibujamos la recta aprendida por el modelo.

Visualmente veremos algo parecido a esto:

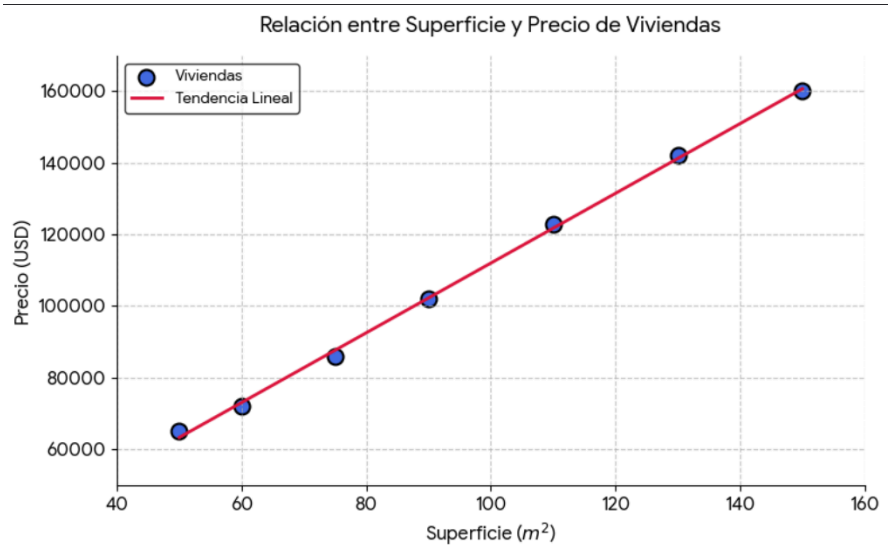


Figura 4.1 — Relación entre superficie y precio de viviendas, con la recta de tendencia lineal.

La recta no necesariamente pasará por todos los puntos, de hecho, sería extraño que lo hiciera, su objetivo es encontrar una relación general.

Esta visualización resulta muy importante pedagógicamente porque permite que el alumno vea aquello que antes solamente habíamos explicado mediante ecuaciones.

La regresión lineal está intentando encontrar **la recta que mejor representa la tendencia general de los datos**.

> *Tenemos diez viviendas y una recta que pasa exactamente por cada uno de los diez puntos. ¿Eso significa necesariamente que encontramos el mejor modelo?*

Si conseguimos una función que reproduce exactamente todos los ejemplos disponibles, podríamos tener un modelo que simplemente memoriza los datos.

Lo que realmente nos interesa es que el modelo funcione sobre ejemplos que todavía no conocemos.

Por eso el aprendizaje automático no se evalúa únicamente por su capacidad para representar los datos históricos.

Se evalúa principalmente por su capacidad de **generalizar**.

4.7. Ampliando el modelo con más variables

4.7.1. Un segundo escenario: cuando incorporamos más información

Nuestro primer modelo utiliza una sola variable:

superficie

Pero sabemos que el precio de una vivienda depende de muchos factores.

Podemos incorporar, por ejemplo:

- superficie;
- cantidad de habitaciones;
- antigüedad;
- distancia al centro;
- cantidad de baños.

Entonces nuestro conjunto de datos podría comenzar a tener esta estructura:

Superficie	Habitaciones	Antigüedad	Distancia	Baños	Precio
80	2	10	4,2 km	1	95.000
110	3	5	2,1 km	2	135.000
150	4	3	1,5 km	2	180.000
70	2	20	6,0 km	1	80.000

Ahora tenemos cinco variables de entrada.

Nuestro modelo ya no puede representarse simplemente como una recta en un gráfico de dos dimensiones, pero la idea fundamental no cambia.

Tenemos:

X=[x1,x2,x3,x4,x5]

y queremos predecir:

y

El modelo aprende un peso para cada variable.

Conceptualmente:

y^=b+w1x1+w2x2+w3x3+w4x4+w5x5

Ahora podemos comenzar a ver por qué hablamos de **Machine Learning multivariable**. El modelo no necesita que nosotros decidamos manualmente cuánto pesa cada característica. Utiliza los datos para aprender esos pesos.

4.7.2. ¿Significa esto que más variables siempre producen un mejor modelo?

No. Y esta es otra pregunta que vale la pena dejar planteada.

Podría parecer lógico pensar:

> *"Si agregamos más información, el modelo necesariamente será mejor."*

Pero no siempre ocurre.

- Una variable puede aportar información útil.
- Otra puede ser irrelevante.
- Otra puede introducir ruido.
- Otra puede estar relacionada de manera engañosa con el objetivo.

Y otra incluso puede contener información que no estaría disponible en el momento en que necesitamos realizar la predicción. Por ejemplo, imaginemos que queremos predecir el precio de una vivienda **antes de venderla**.

Si dentro de nuestro conjunto de datos aparece una variable llamada:

precio_final

esa variable obviamente contiene la respuesta que queremos predecir. Utilizarla como entrada sería absurdo.

El modelo podría obtener resultados extraordinarios durante el entrenamiento, pero estaríamos cometiendo un error conceptual gravísimo.

Este problema se relaciona con lo que más adelante estudiaremos como **data leakage** o fuga de información.

Aunque todavía no necesitamos desarrollar ese concepto, es importante que el estudiante comience a comprender que **no toda información disponible debe convertirse automáticamente en una feature**.

4.7.3. Lo que realmente aprendimos con este ejemplo

Si miramos todo el proceso desde el principio, parece sorprendentemente sencillo.

Comenzamos con datos:

superficie → precio

Definimos:

X → superficie

y → precio

Creamos un modelo:

```
modelo = LinearRegression()
```

Lo entrenamos:

```
modelo.fit(X, y)
```

El algoritmo aprendió sus parámetros.

Después utilizamos:

```
modelo.predict(\...)
```

para generar una predicción sobre una vivienda nueva.

Pero detrás de estas pocas líneas de código existen todos los conceptos que desarrollamos hasta aca:



Ese es exactamente el proceso que quiero que comprendan.Que cuando vean esas tres líneas, puedan explicar **qué problema están resolviendo, qué datos están utilizando, qué está aprendiendo el modelo y cómo determinarían si la solución funciona correctamente**.

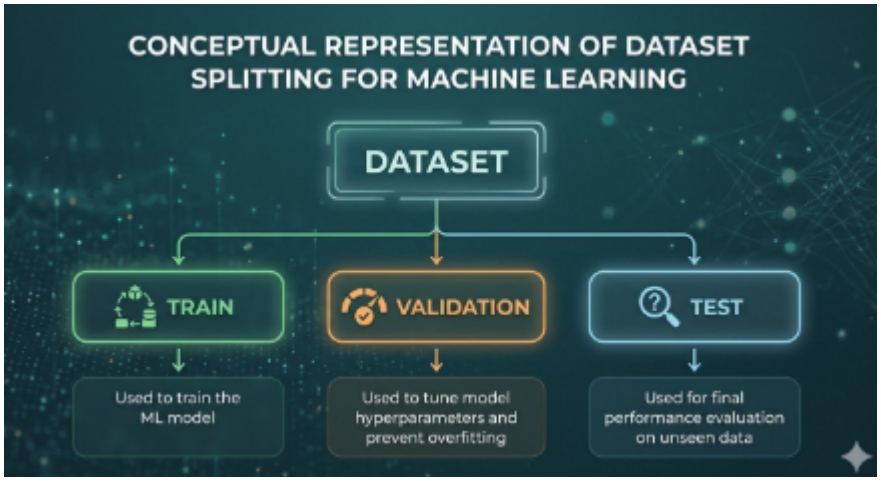
4.7.4. Separar los datos

Supongamos que tenemos 5.000 observaciones. Podríamos utilizar todas ellas para entrenar un modelo.

Pero entonces surge una pregunta:

> ¿Cómo sabemos si funciona bien con datos que nunca vio?

Si evaluamos el modelo sobre los mismos ejemplos que utilizó para aprender, estaremos midiendo principalmente qué tan bien se ajustó a esos datos. No estaremos midiendo adecuadamente su capacidad para generalizar. Por eso necesitamos separar los datos.



- El conjunto de training se utiliza para entrenar.
- El conjunto de validation se utiliza durante el proceso de selección y ajuste del modelo.
- El conjunto de test se reserva para realizar una evaluación final.

La idea fundamental es que el conjunto de prueba debe mantenerse separado del proceso de ajuste. Si miramos repetidamente el test y modificamos nuestro modelo basándonos en sus resultados, estamos utilizando indirectamente esa información para tomar decisiones. En consecuencia, deja de funcionar como una evaluación verdaderamente independiente.

4.7.5. Validación y experimentación

Machine Learning es un proceso experimental.

Rara vez construimos un único modelo, lo entrenamos una vez y concluimos inmediatamente que es la mejor solución.

Podemos tener:

1. Modelo A
2. Modelo B
3. Modelo C

y diferentes configuraciones para cada uno.

Podemos modificar hiperparámetros, características, transformaciones y otros aspectos del proceso.

Pero cada modificación debe evaluarse de manera controlada.

Por ejemplo:

Modelo	Configuración	Validation
KNN	K=3	0,78
KNN	K=7	0,82
Árbol	profundidad=3	0,80
Árbol	profundidad=8	0,76

La tabla no es solamente una forma de registrar resultados. Representa una parte esencial del trabajo científico y técnico con Machine Learning.

Estamos formulando configuraciones, realizando experimentos, observando resultados y tomando decisiones.

Más adelante incorporaremos métricas más completas y analizaremos por qué un modelo con una determinada precisión puede ser menos conveniente que otro.

4.7.6. Cierre

Comenzamos esta unidad con una pregunta aparentemente sencilla: ¿podemos utilizar datos históricos para estimar el valor de una vivienda que todavía no conocemos?

La respuesta nos llevó a construir una regresión lineal.

Primero observamos los datos y buscamos una relación entre una variable de entrada y un valor objetivo. Después representamos esa relación mediante una función matemática y comprendimos que los parámetros de esa función no son elegidos arbitrariamente, sino que deben ser aprendidos a partir de los ejemplos disponibles.

También vimos que aprender no significa memorizar. El verdadero objetivo consiste en encontrar una relación que permita realizar predicciones razonables sobre nuevos casos. Para medir la calidad de esas predicciones necesitamos cuantificar el error y utilizar métricas apropiadas.

Finalmente trasladamos todo ese razonamiento a Python y Scikit-Learn. Vimos que crear un modelo, entrenarlo y utilizarlo para realizar predicciones son operaciones diferentes, aunque en código puedan parecer extremadamente sencillas.

La línea:

```
modelo.fit(X, y)
```

dejó de ser una instrucción misteriosa, ahora sabemos que representa un proceso de aprendizaje.

Y la línea:

```
modelo.predict(X_nuevo)
```

también adquiere un significado mucho más profundo.

Representa el momento en el que utilizamos aquello que el modelo aprendió para enfrentarse a un ejemplo nuevo.

Este recorrido constituye nuestra primera experiencia completa con un algoritmo de Machine Learning. A partir de ahora podremos utilizar esta estructura como referencia para comprender otros modelos y justamente eso será lo que haremos en el futuro.

- Hasta ahora queríamos predecir **un número**.
- Ahora vamos a plantear un problema diferente.
- Una transacción bancaria puede ser **fraudulenta o legítima**.
- Un paciente puede ser clasificado como **de riesgo o sin riesgo**.
- Un correo puede ser identificado como **spam o no spam**.
- Un cliente puede ser clasificado como **propenso a abandonar o no propenso a abandonar**.

Ya no queremos obtener un número. Queremos decidir **a qué categoría pertenece un nuevo ejemplo**, hemos llegado al mundo de la **clasificación**.